Click Montagem — Relatório Técnico

> **Documento:** Auditoria de Código, Correções Funcionais e Migração de Backend
> **De:** PHP 8 + MySQL Relacional (XAMPP/Apache)
> **Para:** Node.js LTS (v24) + Express + MongoDB (Mongoose)
> **Frontend:** HTML5 + CSS3 + Vanilla JavaScript (100% preservado)
> **Status da Validação:** 10/10 Testes de Integração Automatizados Aprovados

1. O Bug Crítico de Troca de Identidade ("João Silva" / "Marcos Oliveira")

Ao realizar cadastro ou login de um novo usuário, a tela frequentemente continuava exibindo o nome de **João Silva** (ou **Marcos Oliveira**). Foram descobertas 4 causas estruturais:

- IDENTIFICADO **Textos Fixos (Hardcoded) no HTML:** Páginas como `painel-cliente.html`, `orcamento.html`, `editar_perfil.html`, `propostas.html`, `painel-montador.html` e `agenda.html` possuíam os nomes "João Silva" (JS) e "Marcos Oliveira" (MO) gravados diretamente nos elementos de navegação e em `<input value="...">`.
- IDENTIFICADO **IDs de Fallback Estáticos no JavaScript:** Em `propostas.js`, quando a URL não continha o ID do orçamento, o script forçava `orcId = 1` (o orçamento de João Silva). Em `avaliacao.js` havia `agendamentoAtualId = 1` e em `perfil-montador.js` havia `montadorId = 2`.
- IDENTIFICADO **Ausência de Sessão no Servidor:** O backend PHP original não gerenciava sessões ativas; confiava cegamente no `localStorage` do navegador, que acumulava dados de contas antigas e não era limpo.
- RESOLVIDO **Correção Implementada:** Implementação de autenticação via sessão segura no servidor (`express-session` com `connect-mongo` em cookies `HttpOnly` e `SameSite=lax`), regeneração de ID de sessão a cada login/cadastro, limpeza automática de cache de armazenamento local legado e obtenção da identidade real através do endpoint central `GET /api/auth/me`.

2. Problemas Encontrados no Frontend e Como Foram Corrigidos

Componente / Arquivo	O que estava errado / Inoperante	O que foi alterado e corrigido
Solicitar Orçamento `pages/orcamento.html`	O botão de envio era um link fixo `<a href="confirmacao-orcamento.html">` que nunca acionava o backend. Faltavam atributos `id` nos campos de endereço e valores nos seletores.	Formulário conectado a `enviarOrcamento()` via API REST. Adicionados IDs (`descricao`, `rua`, `numero`, `bairro`), valores válidos e validação visual de erros.
Confirmação de Orçamento `pages/confirmacao-orcamento.html`	Página 100% estática. Mostrava sempre dados fictícios de um guarda-roupa já preenchido.	Conectada via `GET /api/orcamentos/:id` para carregar e exibir na hora o pedido real recém-criado no MongoDB.

Componente / Arquivo	O que estava errado / Inoperante	O que foi alterado e corrigido
Aceite de Proposta / Modal `pages/propostas.html`	O botão "*Confirmar agendamento*" dentro do modal de data não possuía ação vinculada (clicava e nada acontecia). Faltava folha de estilos do FontAwesome.	Adicionado evento `aceitarProposta()` com integração ao backend (criando o agendamento real e fechando o orçamento) e incluído FontAwesome.
JavaScript Global `assets/js/script.js`	Causava erro fatal no console (`TypeError: null reading addEventListener` na linha 87) em todas as páginas exceto na página inicial.	Adicionada checagem defensiva de existência antes de associar ouvintes de eventos a seletores dinâmicos.
Edição de Perfil `editar_perfil.html` / `editar-perfil-montador.html`	Os campos vinham preenchidos com os dados de exemplo no código-fonte e faltava opção de exclusão.	Removidos valores hardcoded; os formulários agora preenchem com os dados reais do usuário logado via `GET /api/auth/me` e permitem salvar alterações e excluir conta.
Agenda do Montador `pages/agenda.html`	Um bloco de script inline sobrescrevia a função de adicionar disponibilidade com um alerta estático sem persistência.	Removido script duplicado; a agenda agora consome `agenda.js` com dados do banco e atualização real de status.

3. Vulnerabilidades de Segurança do Backend PHP e a Nova API Node.js

- **IDOR (Insecure Direct Object Reference):** No PHP antigo, qualquer usuário podia editar o orçamento, ver propostas ou excluir registros de outra pessoa apenas passando o ID de outro usuário. Na nova API em Node.js, os serviços (`orcamentoService` , `propostaService` , etc.) validam a propriedade do registro com o `req.user._id` da sessão.
- **Validação de Senhas e Redefinição:** No PHP, a rota `redefinir_senha` mudava a senha de qualquer e-mail sem validar o token. No Node.js foi criado fluxo seguro com token criptográfico de uso único (SHA-256) com tempo de expiração de 1 hora.
- **Proteção CSRF e NoSQL Injection:** Adicionado middleware de sanitização recursiva contra operadores maliciosos (`$` e `.`) e proteção CSRF vinculada à sessão em requisições mutáveis (POST/PUT/PATCH/DELETE).
- **Senhas Fortes:** Armazenamento seguro de senhas através de `bcryptjs` com fator de custo 12.

4. Migração e Modelagem MongoDB

O antigo banco relacional MySQL foi transformado em coleções no MongoDB gerenciadas pelo Mongoose:

- `usuarios` **:** Centraliza clientes e montadores. Dados profissionais de montador ficam embutidos no subdocumento `montador` (com índice esparso e único em `cpf_cnpj`).
- `orcamentos` **:** Documentos estruturados com campos de endereço, turno e status de ciclo de vida.

- `propostas`: Possui índice composto exclusivo `{ orcamento_id: 1, montador_id: 1 }`, impedindo propostas duplicadas do mesmo montador no banco.
- `agendamentos`: Com índice único para evitar múltiplos agendamentos concorrentes no mesmo orçamento.
- `avaliacoes`: Vinculadas ao agendamento, permitidas apenas após o status `concluido` e limitadas a 1 por serviço.
- `notificacoes`: Notificações em tempo real sobre propostas recebidas, agendamentos e avaliações.

5. Suíte de Testes Automatizados (100% Aprovada)

Foi desenvolvida uma suíte completa de testes de integração em `tests/api.test.js` executando contra banco isolado (`click_montagem_test`). Todos os 10 cenários foram executados e aprovados:

1. Cadastro de clientes e montadores com bloqueio de e-mail e CPF duplicados.
2. Isolamento rigoroso de sessão e alternância de contas (verificando resolução definitiva do bug de identidade).
3. Login, logout, expiração e recuperação de senha segura com token descartável.
4. Proteção de rotas, autorização de perfis (anti-IDOR) e barreira contra ataques CSRF.
5. Criação, busca e isolamento de orçamentos entre diferentes clientes.
6. Envio de propostas e bloqueio de proposta duplicada no mesmo serviço.
7. Aceite de proposta com recusa automática de concorrentes e criação atômica do agendamento.
8. Ciclo de vida do serviço, proibição de avaliação antecipada e recálculo da nota média do montador.
9. Atualização de perfis cadastrais e marcação de notificações lidas.
10. Regras de cancelamento e integridade de histórico de serviços em andamento.
